

HSG-RAG: Hierarchical Knowledge Base Construction for Embedded System Development

ZHOUYANG LU, School of Computer Science, Fudan University, Shanghai, China

HAILIN XU, School of Computer Science, Fudan University, Shanghai, China

ANRUI CHEN, School of Computer Science, Fudan University, Shanghai, China

SIYUAN TANG, School of Computer Science, Fudan University, Shanghai, China

JUNYI ZHANG, School of Computer Science, Fudan University, Shanghai, China

YIFEI FENG, School of Computer Science, Fudan University, Shanghai, China

WENTAO PAN, School of Computer Science, Fudan University, Shanghai, China

JIANGLI HUANG, Fudan University, Shanghai, China

Customization is a fundamental aspect of embedded system development, requiring developers to acquire extensive domain-specific knowledge from technical documents. However, the sheer volume of these documents, coupled with the intricate relationships between their contents, makes it challenging to efficiently retrieve the necessary information. This challenge highlights the need for a structured approach to organize domain knowledge, with explicit representation of interrelationships. Moreover, while advanced large language models (LLMs) show promise in aiding embedded system development, they often lack the specialized knowledge needed to address domain-specific queries effectively. In this article, we present HSG-RAG, a knowledge base construction and retrieval method tailored for embedded system development that leverages knowledge graphs to represent the hierarchical structure within technical documentation. Unlike prior retrieval-augmented generation (RAG) or GraphRAG-based approaches, which build the index either rely on semantic similarity or keyword co-occurrence, HSG-RAG captures the inherited dependency and hierarchical relationships in the documents, which benefits the retrieval in both performance and efficiency. We also introduce a benchmark for evaluating the effectiveness of RAG systems in solving real-world challenges in embedded systems, particularly for multi-hop question answering. Experimental results show that HSG-RAG outperforms both RAG and GraphRAG, generating more specific and concise responses.

CCS Concepts: • **Information systems** → **Top-k retrieval in databases; Language models;**

Additional Key Words and Phrases: Knowledge base, retrieval augmented generation, large language model, embedded system

This research is supported in part by the National Natural Science Foundation of China under Grant 62090025; and in part by the National Key Research and Development Program of China under Grant 2023YFB4405101.

Authors' Contact Information: Zhouyang Lu, School of Computer Science, Fudan University, Shanghai, China; e-mail: zylu22@m.fudan.edu.cn; Hailin Xu, School of Computer Science, Fudan University, Shanghai, China; e-mail: hlxu24@m.fudan.edu.cn; Anrui Chen, School of Computer Science, Fudan University, Shanghai, China; e-mail: 23110240001@m.fudan.edu.cn; Siyuan Tang, School of Computer Science, Fudan University, Shanghai, China; e-mail: tangsiyuanyuan@163.com; Junyi Zhang, School of Computer Science, Fudan University, Shanghai, China; e-mail: zxtstars@qq.com; Yifei Feng, School of Computer Science, Fudan University, Shanghai, China; e-mail: yffeng22@m.fudan.edu.cn; Wentao Pan, School of Computer Science, Fudan University, Shanghai, China; e-mail: 21110240007@m.fudan.edu.cn; Jiangli Huang (Corresponding author), Fudan University, Shanghai, China; e-mail: 19112020092@fudan.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1084-4309/2025/10-ART105

<https://doi.org/10.1145/3731680>

ACM Reference Format:

Zhouyang Lu, Hailin Xu, Anrui Chen, Siyuan Tang, Junyi Zhang, Yifei Feng, Wentao Pan, and Jiangli Huang. 2025. HSG-RAG: Hierarchical Knowledge Base Construction for Embedded System Development. *ACM Trans. Des. Autom. Electron. Syst.* 30, 6, Article 105 (October 2025), 21 pages. <https://doi.org/10.1145/3731680>

1 Introduction

Customization is a defining characteristic of embedded system development. Each application necessitates the design and implementation of hardware and software tailored to its specific use cases and functional requirements. Consequently, every phase of the development process—from microcontroller selection and hardware optimization to the creation of device drivers—must align with the intended application environment. This customization enhances the efficiency and reliability of embedded systems, enabling them to fulfill their objectives effectively.

To achieve such a level of customization, developers must possess extensive domain knowledge encompassing all aspects of the process, from hardware selection to firmware programming. This includes making hardware-level decisions, such as selecting a microcontroller that meets performance specifications, choosing appropriate peripheral devices, and optimizing circuit layouts, as well as writing efficient drivers, middleware, and application code.

However, extracting this knowledge, often embedded within various forms of documentation, poses significant challenges. First, the volume of documentation—including datasheets, SDK manuals, and other materials—can be overwhelming. Second, the intricate relationships between the information within these documents complicate retrieval, as developers must navigate complex interdependencies. For example, implementing **Real-Time Clock (RTC)** functionality requires accessing RTC pin information from datasheets, retrieving configuration details from reference manuals, and utilizing library functions from SDK documentation. This process involves leveraging causal, sequential, and referential relationships across multiple documents to successfully complete workflows.

Thus, there is a pressing need to organize domain knowledge within these documents in a structured and relationally explicit manner. Our objective is to construct a knowledge base that comprehensively encompasses domain-specific information while supporting effective and efficient retrieval for specific, concise and complete domain knowledge.

Recent advancements in **large language models (LLMs)** [26, 32, 33, 45] have demonstrated their potential to transform various domains, including embedded system development. Developers can query LLMs for solutions to challenges. However, the inherent knowledge of LLMs is often insufficient for tasks demanding specialized customization. Domain-specific information, such as details regarding the configuration of peripheral devices, is often absent from an LLM's training data. Additionally, the extensive size of relevant documents frequently exceeds the LLM's maximum context window, making it impractical to provide the full content directly. These limitations highlight the necessity of constructing a dedicated knowledge base to facilitate domain-specific knowledge retrieval.

Common approaches for knowledge base construction include **knowledge graphs (KG)** [12, 17, 30, 48] and **Retrieval-Augmented Generation (RAG)** [9] methods based on vector databases [28]. Knowledge graphs organize information through defined entities, attributes, and relationships, yet they suffer from high construction costs, poor scalability, and challenges with real-time updates. RAG methods store text as high-dimensional representations, improving retrieval efficiency; however, they are only effective when the answer to a query is located within a specific region of text, and retrieval of that region alone is sufficient to provide the needed information.

A more advanced approach, **Graph Retrieval-Augmented Generation (GraphRAG)** [7], organizes documents into a graph structure, capturing relationships between texts. This method extends traditional RAG techniques, facilitating tasks that require global information such as **query-focused summarization (QFS)**[6]. Nonetheless, GraphRAG remains constrained in fully supporting embedded system development, as it primarily relies on keyword-based knowledge extraction. Many technical documents contain nuances that cannot be captured through keywords alone. For example, representing dependencies between different configuration steps, such as when configuring registers, proves challenging with keyword matching.

In this article, we introduce a knowledge base construction and retrieval method, designated as **Hierarchical Semantic Graph Retrieval Augmented Generation (HSG-RAG)**, tailored for embedded system development. The motivation behind the use of hierarchical structure stems from the fact that engineers typically approach complex tasks through hierarchical decomposition. The hierarchical structure of knowledge aligns well with this methodology, facilitating a more effective representation and retrieval of relevant information. To achieve it, we harness the semantic understanding and information extraction capabilities of LLMs [36, 42], which enables a more comprehensive knowledge extraction that transcends mere keyword identification. A hybrid retrieval approach is also introduced to enhance the efficiency and precision of the retrieval process.

Our contributions are three-fold:

- We introduce HSG-RAG, a knowledge base construction method tailored for embedded system development. This method effectively extracts domain knowledge from extensive document collections and organizes the extracted information into a hierarchical structure that facilitates efficient and precise retrieval.
- We propose a benchmark designed to evaluate the performance of a knowledge base in addressing real-world challenges in embedded system development. This benchmark includes a document corpus sourced from three distinct hardware platforms and three peripheral devices, totaling over 850,000 tokens. We developed 546 question-context-answer pairs for evaluation.
- Experimental results on our benchmark demonstrate the effectiveness of our knowledge graph construction method. With the help of the graph index, HSG-RAG outperforms RAG and GraphRAG to generate a specific and concise response.

2 Related Work

Retrieval-augmented Generation (RAG) RAG enhances LLMs by integrating external knowledge, improving their performance in tasks requiring extensive information. Recent surveys [8, 9, 14, 14, 16, 43, 44] highlight the integration of RAG with retrieval systems to address issues like precision, recall, and coherence. Basic RAG methods often struggle with retrieving accurate and relevant information, leading to potential inaccuracies or biases in generated content [40]. To overcome these challenges, advanced techniques have been developed. These include optimizing indexing through enhanced data granularity and metadata incorporation, as well as query optimization via rewriting and expansion, which refines the initial query for more effective retrieval [24, 31]. Additionally, proactive retrieval strategies like FLARE [19] use predictive modeling to dynamically adjust retrieved information, thus it can better cope with accurate long text generation tasks, while self-memory augmented methods [4] and generation-augmented retrieval [25] refine the retrieval process using generative feedback. Training approaches such as RA-DIT [23] and bridging models [20] further improve the alignment between LLMs and the retrieval component, ensuring a seamless integration.

Graph Retrieval-Augmented Generation (GraphRAG) GraphRAG approaches [29] complement traditional RAG by leveraging structural data, providing an enriched context for LLMs. GraphRAG [7], the first work in this area, is proposed to enhance the capability of RAG approaches to capture global information by organizing data in a graph structure and linking related nodes with hierarchical super nodes, thereby facilitating QFS. The **Graph Retrieval-Augmented Generation (GRAG)** [13] method utilizes k-hop ego-graphs for efficient subgraph retrieval and employs a dual prompting strategy to maintain semantic integrity. HippoRAG [10] introduces a novel framework inspired by human long-term memory, by analyzing multiple connection points in the information network at one time, it can quickly integrate the key information scattered in different data sources in a single query process. **Knowledge Graph Prompting (KGP)** [39] constructs a KG to capture logical connections among documents, enabling an LLM-based traversal agent to navigate and infer answers. Finally, HybridRAG [35] integrates KG-based and vector-based RAG, showcasing the potential of combining these technologies for improved accuracy and relevance in information extraction, particularly in specialized domains such as financial documents.

3 Method

The primary objective of this research is to create an index for a collection of technical documents in the embedded systems field. This index aims at representing the inherent hierarchical relationships between domain knowledge, allowing precise and efficient retrieval for domain-specific queries.

Technical documentation in the embedded systems field contains extensive domain knowledge, including detailed datasheets and SDK manuals that describe parameters, configuration tables, and operational procedures. These documents often reveal complex interdependencies. For example, there are relationships between register configurations and the functionality of specific modules, interactions between SDK function calls and hardware operations, and sequences of configuration steps necessary to implement particular functionalities. To build the index, a knowledge graph is an effective way since it can represent diverse relationships. Its fundamental components are entities and relationships, where each piece of knowledge is expressed as a triple: “<Entity, Relationship, Entity>”. These triples can be combined into chains, allowing the representation of complex semantic relationships between a head entity and a tail entity. On the other hand, it is natural to view the knowledge within a document as a collection of interconnected entities and relationships, and the edges of their respective entities can also represent the associations between documents. Therefore, we design our method based on knowledge graph construction.

We identify two significant challenges in the knowledge base construction process:

— Non-named Entity Extraction

While many named entities can be easily extracted from the text, extracting non-named entities poses a challenge. For instance, in the sentence, “After installing the I2C master driver by `i2c_new_master_bus()`, ESP32-S3 is ready to communicate with other I2C devices,” the named entity “`i2c_new_master_bus`” which represents a “function” can be extracted from the text literally. However, semantic entities such as “Installing I2C Master Driver” and “Communicating with I2C Devices” which represent “tasks” do not literally appear in the original text. Current GraphRAG methods typically extract only keywords associated with named entities, thereby severely limiting the scope of extracted information.

— Cross-Paragraph Relationship Extraction

Entities extracted from different paragraphs often have specific relationships that are challenging to identify. For example, the task “Initializing I2C Master Bus” might be mentioned in the introduction section, while the “`i2c_new_master_bus`” function appears in the API

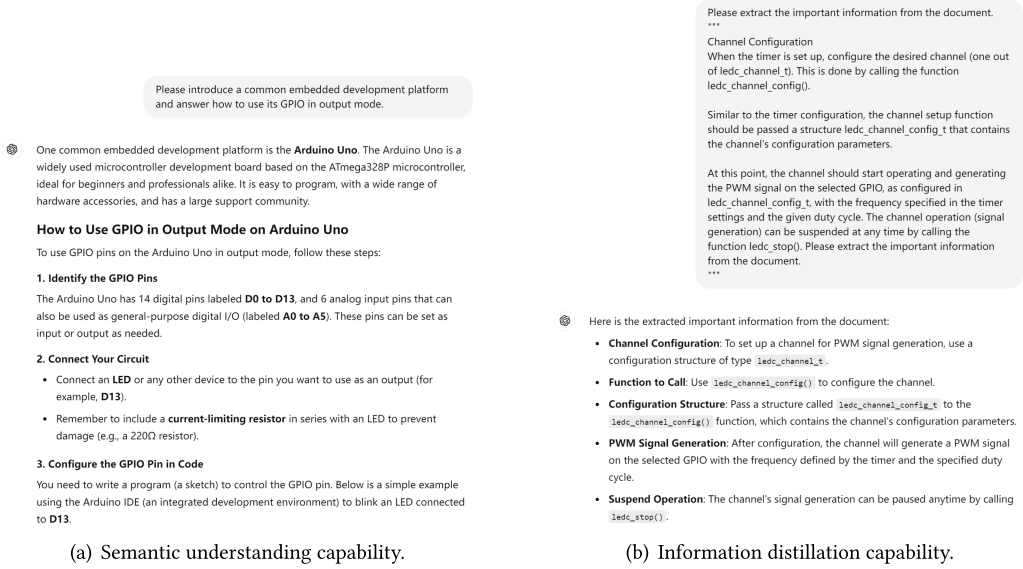


Fig. 1. Capabilities of LLM.

reference section, separated by many paragraphs. These entities share an “IMPLEMENT” relationship. However, in the absence of leveraging the structural hierarchy within these documents, every possible pair of paragraphs must be considered as potentially related. This leads to an exponential increase in the complexity of relationship extraction, rendering it impractical for real-world applications.

We have developed several insights to address these challenges:

- (1) **Leveraging LLMs:** Current LLMs possess strong semantic understanding [27] and information distillation capabilities in the embedded system field, which can facilitate more comprehensive information extraction, including both named and non-named entities. Examples as shown in Figure 1.
- (2) **Reducing Cross-Paragraph Exploring Scope:** Technical documents are inherently written in a structured paradigm. Specifically, we can leverage two features to reduce the exploring scope and build relationships efficiently.
 - (a) **Locality:** The inherent locality of documents indicates that successive paragraphs are more likely related; we refer to relationships extracted within this local context as “short-distance relationships.”
 - (b) **Semantic Similarity:** Non-successive paragraphs with semantically similar entities are more likely to possess semantic relationships, referred to as “long-distance relationships”. Examples as shown in Figure 2.

We designed our method implementation based on the insights from subsection 3.1, consisting of two key components:

— Graph-based Indexing (Subsection 3.1)

This phase aims at building a knowledge base specifically for embedded system development by extracting critical information from documentation and constructing relationships between them. In the knowledge base, each piece of knowledge is represented as a graph node, with relationships depicted as graph edges. As portrayed in Figure 3, the indexing

Timer Configuration

To configure the timer, use `ledc_timer_config()` with the structure `ledc_timer_config_t` that includes parameters like speed mode, frequency, and resolution.

...

Channel Configuration

After setting up the timer, use `ledc_channel_config()` to configure a channel from `ledc_channel_t`, associating it with the timer and setting up GPIO output.

...

Change PWM Signal

Once the channel is active, the duty cycle of the PWM signal can be adjusted to vary LED brightness. This adjustment can be done via software or hardware fading.

...

(a) Short-distance relationships among successive paragraphs.

1.6 Product Picture

The I2C communication interface includes components like the slave address bit (SA0), data signal (SDA), and clock signal (SCL). These signals need pull-up resistors. The SA0 bit helps identify the slave address, with formats like `011110 SA0 R/W#`.

...

3.3.1 I2C Interface Timing Characteristics

This section outlines timing characteristics for the I2C interface, such as clock cycle time (`tCYC`), start condition hold time (`tSTART`), and data hold/setup times. These parameters ensure proper data transfer and signal timing on the I2C bus.

...

(b) Long-distance relationships among non-successive paragraphs.

Fig. 2. Semantic relationships among paragraphs.

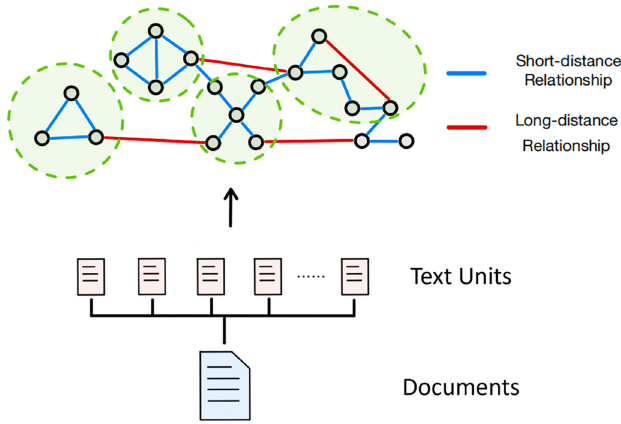


Fig. 3. Graph-based Indexing.

process involves four steps: document preprocessing, knowledge extraction (short-distance knowledge extraction and long-distance knowledge extraction), and graph building.

— Graph-Guided Retrieval (Subsection 3.2)

While the constructed knowledge base facilitates comprehensive knowledge retrieval, the increasing volume of information poses challenges in pinpointing specific insights. Additionally, prior search algorithms have shown suboptimal performance in handling multi-hop queries. To overcome these challenges, we leverage LLMs to decompose user queries into essential subqueries, iteratively explore the graph index, and finally fetch the relevant paragraphs. The retrieval process is illustrated in Figure 4.

3.1 Graph-based Indexing

3.1.1 Document Chunking. Due to the limited text length that LLMs can process at once, it is necessary to divide the document into smaller text units for analysis. The chunking mechanism plays a critical role in determining the efficiency and accuracy of subsequent knowledge extraction processes. It is essential that the selected chunking method preserves the local context, ensuring that each segmented text unit corresponds to a coherent topic. This approach increases the likelihood of higher relevance among consecutive text units. To achieve this, we employ the

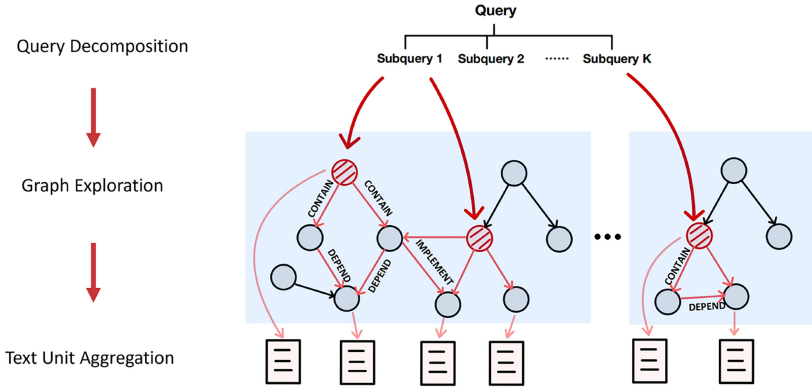


Fig. 4. Graph-guided Retrieval.

LangChain¹ tool to segment the original documents into coherent text chunks that maintain the logical structure of the original material.

3.1.2 Short-distance Knowledge Extraction. We define short-distance knowledge as knowledge that exists within a single text unit or spans across adjacent text units. Knowledge is represented in the form of <Entity, Relationship, Entity> triples. The following outlines their definitions:

- **Entity:** An entity can be a distinct domain-specific named element (e.g., a function, data type, or hardware pin) that is literally mentioned in the text. It can also be some non-named concept such as a system module or task procedure which needs to be summarized from the raw text. Each entity contains a description that indicates its distinct meaning.
- **Relationship:** We define three kinds of relationships between entities, which represent the knowledge dependency in the documentation and are useful during the retrieval process. They are:
 - **DEPEND:** An entity uses or depends on another entity. For example, a function uses a data type, a function calls another function, or an object uses the reference of another object.
 - **CONTAIN:** It represents a combination or aggregation relationship. For example, a system module contains some submodules, or a task can be decomposed into subtasks.
 - **IMPLEMENT:** Represents the realization or execution of a task or functionality by another entity. For example, a task is implemented by a function, or a system module is implemented by a set of functions. It signifies that a specific functionality is carried out through the execution of one or more code components.

Additionally, we define another type of relationship between each entity and the text unit in which it is sourced, including “DEFINE” and “USE”. This information will serve as a basis for ranking text units that correspond to the same entity during retrieval.

To effectively extract the above knowledge, we utilize an LLM to understand semantic meanings and conduct knowledge extraction. Specifically, we organize the prompt with three parts: First, an instruction directs the goal of extracting entities and relationships from the text based on the provided schema. Next, a schema outlines the entities, relationships, and definitions to be extracted. Finally, two examples are provided, each consisting of an input and its corresponding output. The examples work because the LLM is shown to have the ability of in-context learning [1].

¹<https://www.langchain.com/>

Due to the inherent uncertainty in LLM-generated results, it is inevitable that there may be missing knowledge extracted. Some works have demonstrated that LLM shows stronger capabilities in discriminative tasks compared to generative tasks [18, 34]. We implement a reflection mechanism and introduce a “discriminator” agent, which takes the original text, the schema, and the triplet results extracted during the first round as input. The agent is prompted to output a natural language comment on the completeness of the extracted triples and make a judgment on whether a second round of extraction is necessary. If the decision is affirmative, the “extractor” LLM accepts the feedback and updates the extraction results; if not, the extraction process is finished.

3.1.3 Long-distance Knowledge Extraction. In embedded system development, many complex questions cannot be answered by examining a single continuous paragraph. Rather, a comprehensive answer often requires collecting information from different positions. For instance, register definitions may appear early, while detailed configurations and functional descriptions follow later, separated by some irrelevant paragraphs. Consequently, our knowledge graph must not only identify local short-distance knowledge but also capture long-distance knowledge that spans across broader contextual scopes.

A major challenge in identifying long-distance relationships arises from the large number of text units, resulting in a combinatorial explosion of potential relationships. Therefore, a key challenge is to reduce the search space in a computationally efficient manner.

We assert that paragraphs with semantically similar content are more likely to exhibit meaningful relationships, and paragraphs originating from the same document are more likely to be contextually related. To address this, we first calculate the semantic vectors by using an embedding model² on the “description” property of entities extracted during the “Short-Distance Knowledge Extraction” phase. The cosine similarity is then computed between all entities sourced from the same document.

Next, we compute the “**Semantic Intersection-over-Union (Semantic IoU)**” for each paragraph pair, defined as follows:

$$\text{Semantic IoU} = \frac{\sum_{i \in A} \max_{j \in B} (\text{sim}(i, j)) > 0.8 + \sum_{j \in B} \max_{i \in A} (\text{sim}(i, j)) > 0.8}{|A| + |B|}$$

Where:

- A and B represent the sets of entities in the two paragraphs being compared.
- $\text{sim}(i, j)$ denotes the cosine similarity between entity i from set A and entity j from set B .
- The sum is taken over pairs of entities where the cosine similarity exceeds a threshold of 0.8, indicating a potential relationship.
- $|A|$ and $|B|$ represent the number of entities in sets A and B , respectively.

Only paragraph pairs with a Semantic IoU greater than 0.3 undergo knowledge extraction, with the extraction process proceeding similarly to the “Short-Distance Knowledge Extraction”.

By constraining the search to entities within the same document, the overall computational complexity increases linearly with the number of documents, which ensures the scalability of our method.

3.1.4 Graph Building. After completing the knowledge extraction process, the next step involves merging the extracted entities and relationships into a unified graph for efficient retrieval.

Entity Linking. Entity linking is a crucial step as the extracted knowledge is often redundant, with multiple mentions of identical entities across different paragraphs. These entities must be

²<https://platform.openai.com/docs/guides/embeddings/what-are-embeddings>

merged into a single representative entity to avoid redundancy. Entity linking poses a challenge due to instances where the same entity is represented by different names or synonyms (e.g., “GPIO Pin Function Configuration” may refer to different concepts in the hardware and software sections of the document, though they share the same name). Conversely, different names may refer to the same entity (e.g., “Install I2C Master Driver” and “I2C Master Driver Installing Steps” from distinct paragraphs of the same document).

To address these challenges, we employ a hybrid approach combining keyword matching and semantic similarity:

- (1) **Keyword Matching for Named Entities:** Named entities, such as code elements or specific terms, are generally consistent across the document. Therefore, these entities can be merged effectively using keyword matching on their names.
- (2) **Semantic Matching for Non-Named Entities:** For non-named entities, we compute the semantic similarity of their descriptions. If the similarity exceeds a threshold of 0.9, the entities are considered identical.

This hybrid approach balances both efficiency and accuracy in the entity linking process.

Entity Summarization. In this step, we also utilize an LLM to summarize the descriptions of the entities merged in the previous phase. Subsequently, we apply an embedding model to convert the latest description of each entity into a vector, enabling semantic retrieval in the following steps.

Storage. Once the entities are merged into a knowledge graph at the document level, we use the NetworkX³ library to store the graph structure. Additionally, the semantic vectors associated with the entities are stored in an open-source vector database, LanceDB,⁴ which is partitioned by document. Upon completion, the document graph index is fully constructed, as illustrated in Figure 3

3.2 Graph-guided Retrieval

While our knowledge base effectively captures extensive information from documents, harnessing this data requires a high-performance retrieval algorithm. As the volume of stored knowledge grows, so too do the demands on the search algorithm’s efficiency and accuracy. An optimal search method must retrieve all relevant information while filtering out irrelevant data.

Existing RAG methods primarily rely on cosine similarity between the query representation and stored knowledge representations across the entire knowledge base. However, for multi-hop queries that require diverse types of information, relevant data may not exhibit direct semantic similarity to the query but instead possess other meaningful relationships. As a result, relying solely on direct semantic similarity often leads to lower recall. The local and global search approaches in GraphRAG do not fully leverage the graph structure during retrieval, as they only consider nodes within a one-hop range of the entrance node.

To address these challenges, we propose a novel graph-guided hierarchical retrieval algorithm specifically designed to handle multi-hop queries efficiently. This algorithm decomposes user queries, iteratively explores the graph, and ultimately retrieves the relevant paragraphs.

Our retrieval process follows a top-down approach, beginning with the top-level document to select the graph, exploring through the middle-level knowledge graph, and finally reaching the bottom-level text units.

³<https://networkx.org/>

⁴<https://lancedb.com/>

3.2.1 Query Decomposition. In a manner similar to human problem-solving, complex queries often require decomposition to identify the relevant documents for further exploration. To automate this process, we utilize an LLM. The LLM is provided with a title list of indexed documents in the prompt, and it is tasked with determining which documents are pertinent to the query. Subsequently, the LLM generates subqueries for each of the identified relevant documents.

For instance, given the query “How to read the temperature of the DHT11 sensor through GPIO?” and the document list of “stm32_api_reference, DHT11_datasheet, OLED_datasheet”, the LLM, leveraging its foundational knowledge of embedded systems, can easily identify the relevant documents. It then generates subqueries: “How to read data from a GPIO pin?” for the “stm32_api_reference” document and “How to read the temperature data from the sensor?” for the “DHT11_datasheet” document.

The subqueries will be executed in parallel within their respective partitions of the knowledge graph.

3.2.2 Graph Exploration. For each subgraph index, related entities are identified based on the corresponding subquery. The process begins by encoding the subquery using the same embedding model employed during the indexing stage. Subsequently, semantic matching is performed on all entity nodes within the subgraph to retrieve the top k nodes, which are selected as entry nodes.

Next, starting from each entry node, a breadth-first traversal is conducted, following the direct edges to explore all dependent knowledge. This traversal ensures the comprehensive retrieval of relevant information within the subgraph, facilitating efficient knowledge extraction and reasoning.

3.2.3 Text Unit Aggregation. The next step involves obtaining the text units corresponding to the identified entities to supplement the context. Since the same entity may appear in multiple paragraphs, not all paragraphs contribute equally to enriching the entity information. For instance, if the definition of entity A is provided in text unit A , this text unit is more relevant and should take precedence over text unit B if it also mentions entity A .

Following this idea, text units that establish a “DEFINE” relationship with the entity are prioritized. If no such text unit exists, those with a “USE” relationship are considered next.

Finally, the text units extracted from all subgraphs are aggregated and sorted in descending order based on the number of related entities they contain. This ensures that the most relevant and informative text units are given precedence in the context supplementation process.

4 Evaluation

This section evaluates the performance of our RAG system on embedded system developing tasks. To this end, we first constructed a specialized dataset for embedded system development. Then we compare our system with baseline RAG methods. In addition, we do a case study to show our advantages in graph structure.

4.1 Dataset

Although there do exist some public RAG benchmarks, none of them are suitable for our experiment setup. For example, HotpotQA [41], MultiRC [21], and 2WikiMultiHopQA [11] are widely used datasets for open-domain information retrieval, but they are not tailored to the embedded systems domain. While CoIR [22] focuses on repository-level codes and documentation, there is a lack of queries for multi-hop or cross-referencing question answering. Therefore, we construct a dataset sourced from three specific embedded system platforms and three peripherals, consisting of approximately 850 k tokens of documentation and 546 question-context-answer pairs for

Table 1. Dataset Statistics

	Document Type	# Tokens	# Query
ESP32S3 ⁵	SDK Reference	262 k	199
STM32F103 ⁶	SDK Reference	289 k	116
JL701 TWS Earphone Chip ⁷	SDK Reference	258 k	141
DRV10866 Motor Driver ⁸	Datasheet	12 k	30
HS96L03 OLED Display ⁹	Datasheet	9 k	30
DS18B20 Thermometer ¹⁰	Datasheet	23 k	30

Table 2. Templates for Question Synthesis

Type	Template Example	Question Example
API Usage	How to use [API] on the [PLATFORM]?	How to set the parameters for <code>adc_continuous_config</code> on the ESP32-S3?
Platform-Specific Operation	How to do [OPERATION] on the [PLATFORM]?	How can I register event callbacks for ADC continuous mode on ESP32-S3?
Peripheral Control	How to control [PERIPHERAL] to do [OPERATION] via [PROTOCOL]?	How do I configure OLED for sleep mode using I2C?

evaluation. Among these, 37.5% of the pairs are categorized as 2-hop, 23.3% as 3-hop, 19.8% as 4-hop, and 19.4% as 5 or more-hop.

4.1.1 Documentation. We select two popular embedded platforms (i.e., ESP32S3 and STM32F103) and one closed-source platform (i.e., JL701) due to the abundance of available documentation for all three platforms. Additionally, We gather the datasheets of three niche peripherals (i.e., DRV10866 Motor Driver, HS96L03 OLED Display, and DS18B20 Digital Thermometer) to represent real embedded development scenarios. The specific documents that we collected are presented in Table 1.

4.1.2 Queries. Due to the high cost of collecting a large number of question-and-answer pairs with paragraph-level references, we adopted a synthesis approach to generate query pairs.

We conducted an investigation with three experienced embedded system developers to gather real-world issues they encounter during development. These issues are then categorized into three types, including “API Usage”, “Platform-Specific Operation” and “Peripheral Control”. These types range from specific to general in terms of question formulation, and from narrow to broad in the search scope for answers. For each type, we design 5 templates. And for each template we use GPT-4o to generate questions based on the collected documents. Table 2 presents some examples of these templates along with the generated questions.

⁵API Reference - ESP32-S3 - — ESP-IDF Programming Guide v5.3.1 documentation (espressif.com)

⁶https://www.st.com/content/st_com/en/search.html?q=UM1850-t=resources-page=1

⁷TWS earphone series chip - JieLi Inc.

⁸DRV10866 data sheet, product information and support | TI.com

⁹HS96L03W2C03 HS | Displays - LCSC Electronics

¹⁰DS18B20 TO-92 Digital Thermometer Temperature IC Sensor - Parallax

4.2 Baselines

To evaluate the effectiveness of our proposed method, we conduct a comparative analysis with 2 baseline approaches: the conventional RAG method, and one of the latest Graph RAG-based methods proposed by Microsoft [7]. The experimental settings for the baseline methods are as follows:

- **VanillaRAG:** We adopted a widely used configuration by setting the chunk size to 512. We conducted tests using industry-leading embedding models, including the proprietary model OpenAI’s text-embedding-3-small and the open-source model BAAI’s bge-m3 [3].
- **GraphRAG:** For GraphRAG, we followed its default configuration, setting the chunk size to 600. We evaluated both its Local Search and Global Search methods.

4.3 Metrics

Response Generation. The evaluation of natural language generation quality is challenging, as it is difficult to provide definitive answers, and the standards for assessment are subjective and vary across different domains and design goals [2, 15]. Therefore, customized human evaluation metrics like relevance [47], fluency [38], and diversity [37] are often used.

In the field of embedded system development, questions typically have multiple solutions, making answers non-unique and open-ended. Furthermore, tutorial-style responses often have a hierarchical and complex structure, allowing for further refinement and optimization. Based on these characteristics of embedded systems question-answering, we have established the following evaluation criteria:

- **Specificity:** The degree to which an answer provides precise and detailed information, such as specifying exact functions, methods, or steps, rather than relying on general or vague descriptions.
- **Conciseness:** The extent to which an answer directly addresses the question, avoiding unnecessary or irrelevant information, while maintaining clarity and relevance.
- **Completeness:** The level of coverage an answer provides, ensuring all critical aspects, steps, and relevant details needed to fully address the question are included.

Conducting human evaluations of each output result is both resource-intensive and susceptible to bias. However, numerous studies have demonstrated that LLMs exhibit human-level consistency in evaluating natural language generation tasks [5, 46]. Drawing on these methods, we design an LLM-based evaluator that assesses answer quality according to the above criteria. Specifically, the evaluator is provided with the question, a reference answer, target metrics, and a pair of responses generated by two different methods. It is then tasked with determining the winner or if the responses result in a tie.

Information Retrieval. We assess the retrieval performance using recall@top k. Specifically, we utilize recall@5, and recall@7, recall@10, and recall@20 (R@5, R@7, R@10, and R@20) in our experiments.

4.4 Results

From the knowledge graph constructed by our method, it is evident that we have successfully captured hierarchical domain knowledge and established meaningful associations. This enables HSG-RAG to achieve superior results compared to baseline methods in domain knowledge question-answer tasks, particularly in terms of the specificity and conciseness of the answers. Through ablation experiments, we further confirmed that our hierarchical graph retrieval method outperforms embedding-based matching in terms of recall.

Table 3. Statistics on Graphs Created by HSG-RAG

	ESP32S3	STM32F103	JL701	DRV10866	HS96L03	DS18B20
# of Tokens	262 k	289 k	258 k	12 k	9 k	23 k
# of Text Units	1,337	1,395	1,130	45	24	81
# of Nodes	5,316	6,867	4,924	176	163	356
#of Relationships	8,619	11,632	4,761	80	117	340

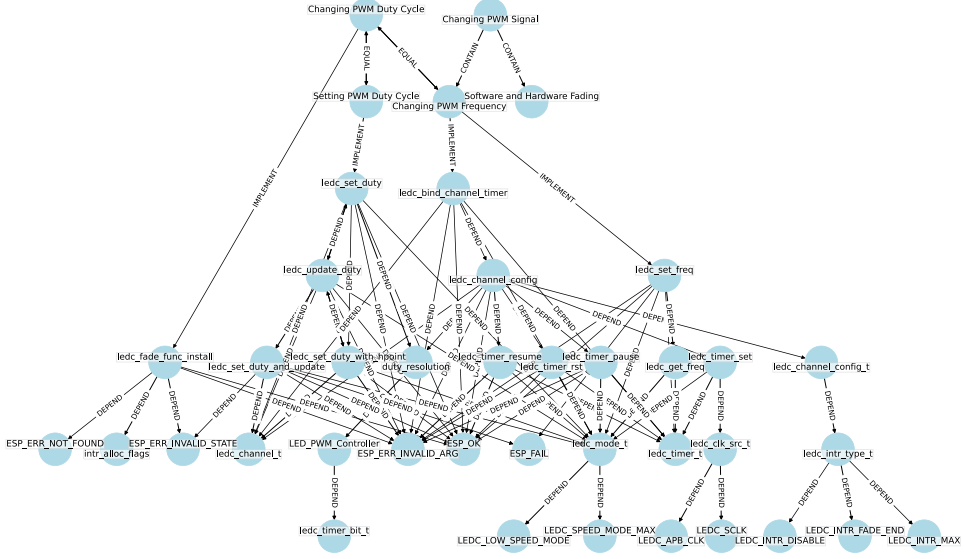


Fig. 5. Visualization of the knowledge graph constructed by HSG-RAG.

4.4.1 Knowledge Graph Result. As shown in Table 3, HSG-RAG extracts a total of 17,802 entities and 25,549 relationships from the corpus, effectively organizing domain-specific knowledge into a graph structure. As shown in Figure 5, the hierarchical relationship between the tasks “Changing PWM Signal” and “Changing PWM Frequency” is clearly represented in the graph. Additionally, the dependency relationships between code elements are captured, such as the need to call “ledc_update_duty” after invoking the function “ledc_set_duty” to update the settings, and the function “ledc_timer_set” requires a parameter of type “ledc_clk_src_t”. Furthermore, the graph illustrates the “IMPLEMENT” relationship between the task “Setting PWM Duty Cycle” and the function “ledc_set_duty”. This relationship highlights how higher-level tasks in the system are implemented using specific functions, bridging the gap between high-level tasks and their corresponding low-level code implementations. By representing these connections, the graph facilitates a better understanding of the system’s architecture, making it easier to navigate complex code bases and identify relevant information at different levels of granularity.

4.4.2 Response Generation Performance. As shown in Figures 4 and 6, our method, HSG-RAG, outperforms the baseline methods across the all of the metrics which are specificity, conciseness, and completeness. Notably, HSG demonstrates a significant in conciseness, with a win rate ranging from 63% to 77%. Our method efficiently filters out irrelevant information, presenting answers that are direct and to the point, without sacrificing necessary details. In contrast, VanillaRAG sometimes includes superfluous information, while GraphRAG’s answers, particularly in its Local

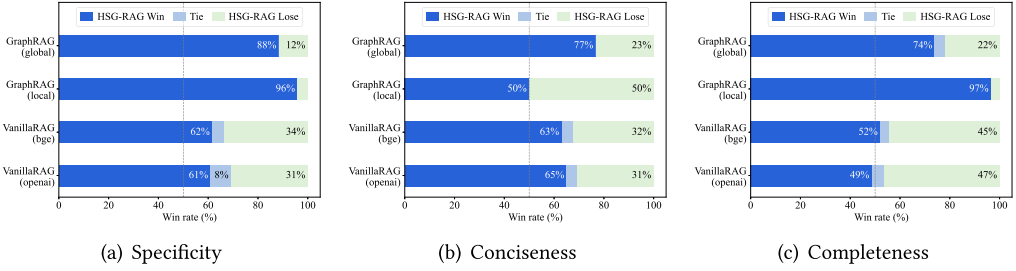


Fig. 6. Win rate percentages of HSG-RAG compared to each baseline method in terms of specificity, conciseness, and completeness.

Table 4. Retrieval Performance for Different Search Configurations

Setting	Entry Num	Max Depth	R@5	R@7	R@10	R@20
embedding-only	-	-	45.1	52.1	59.4	75.2
hierarchical-e1-d3	1	3	41.2	46.9	49.3	54.8
hierarchical-e3-d3	3	3	49.8	57.9	63.4	71.7
hierarchical-e5-d3	5	3	50.4	58.1	66.8	76.8
hierarchical-e7-d3	7	3	50.9	58.9	67.5	80.5
hierarchical-e9-d3	9	3	50.9	58.3	66.3	80.1
hierarchical-e7-d1	7	1	51.2	59.0	67.3	79.4
hierarchical-e7-d2	7	2	50.9	58.9	67.5	80.2
hierarchical-e7-d4	7	4	50.9	58.9	67.5	80.5

Search configuration, tend to be overly verbose. This makes the responses from HSG not only more efficient but also easier to understand, which is critical in embedded systems development where users often seek quick and actionable information.

In terms of specificity, HSG achieves a win rate of 61% to 96%, indicating its ability to identify more specific details and provide more precise answers. This is largely due to our method's ability to incorporate hierarchical and structured knowledge, which helps in retrieving highly specific information from documents.

4.4.3 Information Retrieval Performance. To demonstrate the advantages of our hierarchical graph-guided retrieval approach, we conduct an ablation study focusing on three key aspects: the search method, the number of entry nodes, and the maximum exploration depth of the graph. The results are presented in Table 4.

In general, the hierarchical search outperforms the embedding search baseline, particularly when the top k is small. For instance, hierarchical-e7-d1 configuration performs 6.1% better than the embedding-only method on R@5, and 6.9% better on R@7. This indicates that the graph index is effective in connecting relevant information without introducing similar but irrelevant data.

The best performance was achieved when the entry number is set to 7. This configuration strikes an optimal balance between the breadth and depth of the search space. Regarding the maximum exploration depth, performance saturation was observed at a depth of 3, where further increases in depth did not yield significant improvements. Therefore, the optimal setting was determined to be hierarchical-e7-d3, which provides the best retrieval performance while avoiding unnecessary computational complexity.

Query: How can I initialize and control OLED to set COM output scan direction via I2C?		
HSG-RAG: To initialize and control an OLED display to set the COM output scan direction via I2C, you need to follow these steps: **I2C Device Configuration**: First, configure the I2C device using the <code>i2c_device_config_t</code> structure. You need to set the device address, address length, and SCL speed. For example: <pre> i2c_device_config_t dev_cfg = { .dev_addr_length = I2C_ADDR_BIT_LEN_7, .dev_addr = 0x3C, // Default IIC Address for OLED .scl_speed_hz = 100000, }; </pre> **Add Device to I2C Bus**: After configuring the device, add it to the I2C bus using <code>i2c_master_bus_add_device()</code>: <pre> i2c_master_dev_handle_t dev_handle; ESP_ERROR_CHECK(i2c_master_bus_add_device(I2C_PORT_NUM, &dev_cfg, &dev_handle)); </pre> **Set COM Output Scan Direction**: To set the COM output scan direction, you need to send the appropriate command to the OLED. The command for setting the COM output scan direction is '0xC8' for normal mode (RESET) or '0xC0' for remapped mode. This is detailed in text snippet <code><<1>></code>. You can use <code>i2c_master_transmit()</code> to send this command: <pre> uint8_t command[2] = {0x00, 0xC8}; // Control byte followed by command ESP_ERROR_CHECK(i2c_master_transmit(dev_handle, command, sizeof(command), -1)); </pre> **Additional Initialization**: Make sure to perform any additional initialization steps required for your specific OLED display, such as setting the display on, configuring contrast, etc.	VanillaRAG: To initialize and control an OLED display to set the COM output scan direction via I2C, you need to follow these steps: **I2C Initialization**: Set up the I2C communication interface by configuring the I2C master bus. This includes defining the clock source, I2C port, and the SDA/SCL pins. You can use a configuration structure similar to the one shown in text snippet <code><<7>></code>. **Set Slave Address**: The OLED display typically has a default I2C address (e.g., 0x3C). You need to ensure that your I2C master recognizes this address before transmitting any commands (as described in text snippets <code><<4>></code> and <code><<6>></code>). **Send Initialization Commands**: After establishing the I2C connection, you can send the necessary commands to initialize the OLED. To set the COM output scan direction, you will send the command '0xC8' for normal mode (RESET) or '0xC0' for remapped mode. This is detailed in text snippet <code><<1>></code>. **Control Display Settings**: You can also send other commands to control the display settings, such as turning the display on or off, setting the contrast, etc., as outlined in text snippets <code><<5>></code> and <code><<2>></code>. **Example Command Sequence**: An example command sequence to set the COM output scan direction might look like this: - Send command '0xC8' to set the scan direction from COM0 to COM[N-1]. - Send command '0xAF' to turn the display on.	GraphRAG (Global Search): To initialize and control an OLED display to set the COM output scan direction via I2C on the ESP32-S3, you will need to follow a series of steps that involve configuring the I2C communication settings, sending specific commands to the OLED, and ensuring proper initialization. Below is a comprehensive guide based on the relevant information. ### Step 1: Configure I2C Communication **Set Up I2C Parameters**: You must configure the I2C communication settings, including the I2C address of the OLED display and the GPIO pins for SDA and SCL. This setup is crucial for establishing a successful communication link between the ESP32-S3 and the OLED display [Data: Reports (1, 2, 3, 4, 5, +more)]. **Initialize I2C Bus**: Use the <code>I2C_NEW_MASTER_BUS</code> function to initialize the I2C master bus with the required configurations. This function allocates resources and sets up the bus for communication [Data: Reports (614, 384, 409, 461, 415)]. **Configure GPIO Pins**: Ensure that the GPIO settings for SDA and SCL are correctly configured. This is essential for reliable I2C communication [Data: Reports (260, +more)]. ### Step 2: Initialize the OLED Display **Send Initialization Commands**: After establishing I2C communication, you will need to send initialization commands to the OLED display. This typically includes setting the display mode, resolution, and other parameters as specified in the OLED's datasheet [Data: Reports (598)]. **Set COM Output Scan Direction**: To set the COM output scan direction, you will need to send specific commands to the OLED display. These commands are defined in the OLED's datasheet and may involve writing to the control registers of the OLED. The command for setting the scan direction must be sent after the display has been initialized [Data: Reports (58, 289, 1075, 1074, 1081, 1084, +more)]. ### Step 3: Sending Commands **Use I2C Functions**: Utilize the I2C functions to send the necessary commands to the OLED. The <code>I2C_MASTER_TRANSMIT</code> function is responsible for transmitting data to the OLED's I2C address, ensuring that the commands are correctly sent over the I2C bus [Data: Reports (615)]. **Error Handling**: Implement error handling mechanisms to manage potential issues during the initialization and command sending process. This includes checking for return values from I2C functions to ensure successful communication [Data: Reports (614, 384, 409, 461, 415)].

Fig. 7. Comparison of responses.

4.4.4 Case Study. An example from the evaluation results is presented in Figure 7, where we compare the responses generated by HSG with those generated by VanillaRAG and GraphRAG. The comparison highlights the distinct advantages of our hierarchical semantic graph index.

Among all, HSG-RAG generates highly specific, step-by-step responses. Each step in the response directly addresses the API usage on the ESP32S3 platform, with precise details regarding the relevant parameters. Moreover, the parameter values provided in the response align perfectly with the data described for the OLED module in the manual, ensuring consistency. This enables users to seamlessly apply the sample code directly from the response without the need to consult the manual or any additional resources. The response is structured in a way that allows for immediate practical use, enhancing the user experience.

On the other hand, VanillaRAG offers a correct overall description of the steps involved but lacks the specificity and detail provided by HSG-RAG. While it accurately outlines the general steps, it does not include concrete code examples, which could limit its usefulness for users seeking immediate implementation. Additionally, VanillaRAG introduces some extraneous information that is not directly relevant to the query (e.g., “You can also send other commands to control the display settings, such as turning the display on or off, setting the contrast.”), potentially confusing the user and detracting from the clarity of the response.

GraphRAG, while comprehensive in its scope, falls short in terms of specificity. The description is broad and covers the entire process but lacks the fine-grained details necessary for immediate application. For example, it mentions “To set the COM output scan direction, you will need to send specific commands to the OLED display.” but it doesn’t tell what the specific command is. Consequentially, users would still need to refer to the original manual or other external resources to fill in the gaps, which detracts from the efficiency of the retrieval. Despite being thorough, the lack of specificity makes it less user-friendly for those looking for targeted, actionable answers.

Table 5. Overhead Statistics

	Prompt Tokens	Completion Tokens	Prices (for gpt-4o-mini, USD)
Index Phase (1-million-tokens level)			
Document Chunking	-	-	-
Short-Distance Knowledge Extraction	2.86 m	956 k	1.02
Long-Distance Knowledge Extraction	3.22 m	444 k	0.76
Graph Building	172 k	10 k	0.04
Retrieval Phase			
Query Decomposition	438	69	0.001

5 Discussion

5.1 Efficiency in Graph Construction

In addition to the effectiveness and performance, the efficiency and scalability of the method are crucial factors influencing my method's usability. The primary overhead in a RAG system is concentrated in the offline indexing and online retrieval phases. We calculate the overhead of during our experimental process and estimate the amount of overhead required for a document library at the 1-million-token level. The results are illustrated in Table 5.

5.2 Correctness of Generated Responses

Correctness of Generated Responses While tutorial-like responses generated by LLMs can provide inspiration and guidance for users, the code examples generated are not necessarily correct or executable. Moreover, the lack of an effective verification mechanism makes it difficult for users to assess the correctness of the content, meaning that users need to validate and test the code during actual use to ensure its applicability and alignment with their needs. Future research directions could include developing automatic code validation and correction systems to assist users in verifying the feasibility of generated content and providing guidance for error detection and correction, thereby enhancing the user experience.

5.3 Diversity of Datasets

Due to the high cost of data collection, existing datasets are limited in their ability to cover various platforms and devices comprehensively, restricting the diversity of the evaluation datasets. In addition, the quantity of multi-hop question-answering data in real-world scenarios is relatively small, and the data distribution is not sufficiently comprehensive, which limits the system's generalization ability across different scenarios and makes it difficult for evaluation results to fully reflect performance in real-world applications. Future research should expand the scope of datasets, increasing the representation of real-world data from multiple platforms and devices, and enhancing the volume and diversity of multi-hop question-answering data to improve the system's generalization capabilities across different scenarios and the representativeness of evaluation.

6 Conclusion

To the best of our knowledge, this is the first exploration and discussion of a knowledge base construction method aimed at enabling LLMs to efficiently, comprehensively, and precisely retrieve domain-specific knowledge in the field of embedded system development. Our approach emphasizes the inherent locality of documents as a critical factor in efficiently extracting relevant domain knowledge from vast collections of technical documents. Additionally, we recognize the inherited dependency and hierarchical relationships between the domain knowledge, beyond simple semantic similarity, which previous methods have overlooked, leading to poor performance in multi-hop retrieval scenarios. We propose a new benchmark for evaluating domain-specific question-answer task for embedded system development, assessing the response quality from the

aspects of specificity, conciseness and completeness. Our experimental results demonstrate that the knowledge base built using our method, combined with our graph-guided retrieval algorithm, achieves superior retrieval and response generation performance. Although still in its infancy, we believe our work marks a significant step toward the automated development of embedded systems utilizing LLMs.

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901.
- [2] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.
- [3] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics ACL 2024*. 2318–2335.
- [4] Xin Cheng, Di Luo, Xiuying Chen, Lemao Liu, Dongyan Zhao, and Rui Yan. 2023. Lift yourself up: Retrieval-augmented text generation with self-memory. *Advances in Neural Information Processing Systems* 36 (2023), 43780–43799.
- [5] Cheng-Han Chiang and Hung-Yi Lee. 2023. Can large language models be an alternative to human evaluations?. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 15607–15631.
- [6] Hoa Trang Dang. 2006. DUC 2005: Evaluation of question-focused summarization systems. In *Proceedings of the Workshop on Task-Focused Summarization and Question Answering*. Tat-Seng Chua, Jade Goldstein, Simone Teufel, and Lucy Vanderwende (Eds.), Association for Computational Linguistics, Sydney, Australia, 48–55. Retrieved from <https://aclanthology.org/W06-0707>
- [7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [8] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6491–6501.
- [9] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2 (2023).
- [10] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [11] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, Barcelona, Spain (Online), 6609–6625. Retrieved from <https://www.aclweb.org/anthology/2020.coling-main.580>
- [12] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia D’amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. 2021. Knowledge graphs. *ACM Computing Surveys* 54, 4 (2021), 37 pages. DOI: <https://doi.org/10.1145/3447772>
- [13] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. 2024. Grag: Graph retrieval-augmented generation. *arXiv preprint arXiv:2405.16506* (2024).
- [14] Yucheng Hu and Yuxing Lu. 2024. RAG and RAU: A survey on retrieval-augmented language model in natural language processing. *arXiv preprint arXiv:2404.19543* (2024).
- [15] Minlie Huang, Xiaoyan Zhu, and Jianfeng Gao. 2020. Challenges in building intelligent open-domain dialog systems. *ACM Transactions on Information Systems* 38, 3 (2020), 1–32.
- [16] Yizheng Huang and Jimmy Huang. 2024. A survey on retrieval-augmented text generation for large language models. *arXiv preprint arXiv:2404.10981* (2024).
- [17] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S. Yu. 2022. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems* 33, 2 (2022), 494–514. DOI: <https://doi.org/10.1109/TNNLS.2021.3070843>

- [18] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Houda Bouamor, Juan Pino, and Kalika Bali (Eds.), Association for Computational Linguistics, Singapore, 1827–1843. DOI : <https://doi.org/10.18653/v1/2023.findings-emnlp.123>
- [19] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 7969–7992.
- [20] Zixuan Ke, Weize Kong, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. 2024. Bridging the preference gap between retrievers and LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.), Association for Computational Linguistics, Bangkok, Thailand, 10438–10451. DOI : <https://doi.org/10.18653/v1/2024.acl-long.562>
- [21] Daniel Khashabi, Snigdha Chaturvedi, Michael Roth, Shyam Upadhyay, and Dan Roth. 2018. Looking beyond the surface: A challenge set for reading comprehension over multiple sentences. In *Proceedings of the North American Chapter of the Association for Computational Linguistics*.
- [22] Xiangyang Li, Kuicai Dong, Yi Quan Lee, Wei Xia, Hao Zhang, Xinyi Dai, Yasheng Wang, and Ruiming Tang. 2024. Coir: A comprehensive benchmark for code information retrieval models. *arXiv preprint arXiv:2407.02883* (2024).
- [23] Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Richard James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, et al. 2023. RA-DIT: Retrieval-augmented dual instruction tuning. In *The Twelfth International Conference on Learning Representations*.
- [24] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting in retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 5303–5315.
- [25] Yuning Mao, Pengcheng He, Xiaodong Liu, Yelong Shen, Jianfeng Gao, Jiawei Han, and Weizhu Chen. 2021. Generation-augmented retrieval for open-domain question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.), Association for Computational Linguistics, Online, 4089–4100. DOI : <https://doi.org/10.18653/v1/2021.acl-long.316>
- [26] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2024. Large language models: A survey. <https://doi.org/10.48550/arXiv.2402.06196>
- [27] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an LLM to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal). Association for Computing Machinery, New York, NY, USA, Article 97, 13 pages. DOI : <https://doi.org/10.1145/3597503.3639187>
- [28] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *The VLDB Journal* 33, 5 (2024), 1591–1615.
- [29] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921* (2024).
- [30] Ciyuan Peng, Feng Xia, Mehdi Naseriparsa, and Francesco Osborne. 2023. Knowledge graphs: Opportunities and challenges. *Artificial Intelligence Review* 56, 11 (2023), 13071–13102.
- [31] Wenjun Peng, Guiyang Li, Yue Jiang, Zilong Wang, Dan Ou, Xiaoyi Zeng, Derong Xu, Tong Xu, and Enhong Chen. 2024. Large language model based long-tail query rewriting in taobao search. In *Companion Proceedings of the ACM Web Conference 2024*. 20–28.
- [32] Libo Qin, Qiguang Chen, Xiachong Feng, Yang Wu, Yongheng Zhang, Yinghui Li, Min Li, Wanxiang Che, and Philip S. Yu. 2024. Large language models meet nlp: A survey. *arXiv preprint arXiv:2405.12819* (2024).
- [33] Mohaimenul Azam Khan Raiaan, Md. Saddam Hossain Mukta, Kaniz Fatema, Nur Mohammad Fahad, Sadman Sakib, Most Marufatul Jannat Mim, Jubaer Ahmad, Mohammed Eunus Ali, and Sami Azam. 2024. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE Access* 12 (2024), 26839–26874. DOI : <https://doi.org/10.1109/ACCESS.2024.3365742>
- [34] Matthew Renze and Erhan Guven. 2024. Self-reflection in LLM agents: Effects on problem-solving performance. *arXiv preprint arXiv:2405.06682* (2024).
- [35] Bhaskarjit Sarmah, Dhagash Mehta, Benika Hall, Rohan Rao, Sunil Patel, and Stefano Pasquali. 2024. Hybridrag: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. In *Proceedings of the 5th ACM International Conference on AI in Finance*. 608–616.
- [36] Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. 2023. Enhancing knowledge graph construction using large language models. *arXiv preprint arXiv:2305.04676* (2023).
- [37] Chris Van Der Lee, Albert Gatt, Emiel Van Miltenburg, Sander Wubben, and Emiel Krahmer. 2019. Best practices for the human evaluation of automatically generated text. In *Proceedings of the 12th International Conference on Natural Language Generation*. 355–368.

- [38] Hao Wang, Bin Guo, Jiaqi Liu, Yasan Ding, and Zhiwen Yu. 2023. Towards informative and diverse dialogue systems over hierarchical crowd intelligence knowledge graph. *ACM Transactions on Knowledge Discovery from Data* 17, 7 (2023), 1–25.
- [39] Yu Wang, Nedim Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. 2024. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence* 38 (2024), 19206–19214.
- [40] Shangyu Wu, Ying Xiong, Yufei Cui, Haolun Wu, Can Chen, Ye Yuan, Lianming Huang, Xue Liu, Tei-Wei Kuo, Nan Guan, et al. 2024. Retrieval-augmented generation for natural language processing: A survey. *arXiv preprint arXiv:2407.13193* (2024).
- [41] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [42] Liang Yao, Jiazhen Peng, Chengsheng Mao, and Yuan Luo. 2025. Exploring large language models for knowledge graph completion. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.
- [43] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. 2024. Evaluation of retrieval-augmented generation: A survey. In *CCF Conference on Big Data*. Springer, 102–120.
- [44] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2024. Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473* (2024).
- [45] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2024. A Survey of Large Language Models. *arXiv:2303.18223*. Retrieved from <https://arxiv.org/abs/2303.18223>
- [46] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems* 36 (2023), 46595–46623.
- [47] Ming Zhong, Yang Liu, Da Yin, Yuning Mao, Yizhu Jiao, Pengfei Liu, Chenguang Zhu, Heng Ji, and Jiawei Han. 2022. Towards a unified multi-dimensional evaluator for text generation. *arXiv:2210.07197*. Retrieved from <https://arxiv.org/abs/2210.07197>
- [48] Xiaohan Zou. 2020. A survey on application of knowledge graph. *Journal of Physics: Conference Series* 1487, 1 (2020), 012016. DOI : <https://doi.org/10.1088/1742-6596/1487/1/012016>

Appendix

A1 LLM Prompts

The prompts we used for indexing, query and evaluation are shown in Figure A2, Figure A3, Figure A4 and Figure A1.

```

You are an expert in embedded system skilled in domain
specific customization. You are knowledgeable about general
domain knowledge about RTOS, MCU, etc.

-Goal-
Given a question with a reference answer, and two responses
written by two people, your goal is to evaluate the
responses and select the better one. You should read the
criteria carefully and for each metric, decide which
response is better or tie.

-Criteria-
- Specificity: Identify which answer is more specific, such
as explicitly mentioning the exact functions, commands, or
steps required, rather than providing a vague or general
description.
- Conciseness: Determine which response directly addresses
the question in a clear and focused manner, avoiding
unnecessary or unrelated information.
- Completeness: Assess which answer provides a more
comprehensive explanation, covering all the key points
necessary to fully resolve the question.

-Output Format-
{
  "title": "ResponseEvaluation",
  "type": "object",
  "properties": {
    "specificity": {"type": "string", "enum": ["A", "B",
    "Tie"]},
    "conciseness": ...,
    "completeness": ...
  }
}

-Question-
{{question}}

-Reference Answer-
{{reference_answer}}

-Response A-
{{response_a}}

-Response B-
{{response_b}}

```

Fig. A1. Prompt for the response evaluator.

```

You are an embedded system developer skilled in domain
specific customization. You are knowledgeable about general
domain knowledge about RTOS, MCU, etc.

-Goal-
Given a technical document, first identify all entities
followed by the defined types. For each entity, extract the
entity name, type, document relationship and description.
Return output in English as a single list of all the
entities. Use ## as the list delimiter. When finished,
output <|COMPLETE|>

-Schema-
For Code Entity:
- entity_name: The literal name mentioned in the document.
Must be exactly the same as the name in the document.
- entity_type: Suggest the categories for the entity, one
of ["FUNCTION", "STRUCT", "MACRO", "ENUM", "FILE_NAME",
"PIN", "REGISTER_NAME", "DATA_TYPE", "UNKNOWN_NAME"].
- document_relation: The relationship between the entity
and the document, one of ["DEFINE", "USE"].
- entity_description: The summarized content of the piece
of knowledge, each piece of knowledge should be complete
sentences.
Format each entity as
("code"<|><entity_name><|><entity_type><|><document_rel
ation><|><entity_description>)

For Semantic Entity:
- entity_name: A noun phrase or a verb phrase that
represents a concept or a task.
- entity_type: Suggest the categories for the entity, one
of ["TASK", "SYSTEM_MODULE", "OTHER"].

- document_relation: The relationship between the entity
and the document, one of ["DEFINE", "USE"].
- entity_description: The summarized content of the piece
of knowledge, each piece of knowledge should be complete
sentences.
Format each entity as
("semantic"<|><entity_name><|><entity_type><|><document_rel
ation><|><entity_description>)

-Examples-
input document:
esp_err_t esp_spp_init(esp_spp_mode_t mode)
This function is called to ...
output:
("code"<|>esp_spp_init<|>FUNCTION<|>DEFINE<|>This is a
function called to initialize the SPP module.)##
...
("semantic"<|>Initializing SPP module<|>TASK<|>USE<|>This
represents the process of initializing the SPP module.)##
<|COMPLETE|>

-Real Data-
input document: {{input}}
output:

```

Fig. A2. Prompt for entity extraction.

```

You are an embedded system developer skilled in domain
specific customization. You are knowledgeable about general
domain knowledge about RTOS, MCU, etc.

-Goal-
Given a technical document with a set of entities, identify
the dependencies between the entities and return the output.
For each relationships, extract the source, type and target.
Return output in English as
(<source><|><relationship><|><target>). Use ## as the list
delimiter. When finished, output <|COMPLETE|>

-Example 1-
-Input Document-
esp_err_t esp_spp_init(esp_spp_mode_t mode)
This function is called to ...

-Entity Set-
(code<|>esp_spp_init<|>FUNCTION<|>This is a function called
to initialize the SPP module.)##
...
(semantic<|>Initializing SPP module<|>TASK<|>This
represents the process of initializing the SPP module.)##

-Schema-
There are two types of entities: code entities and semantic
entities. Code entities are code elements such as functions,
structs, macros, enums, or header files in the C language,
and pins, registers in the hardware datasheet. Semantic
entities are semantic concepts such as tasks or system
modules.

The relationship types include:
- USE: <code> -USE-> <code>: A code entity uses another
code entity. For example, a function uses a data type, a
function call another function, a struct contains a member
of another struct, etc.
- CONTAIN: <semantic> -CONTAIN-> <semantic>: A system
module contains some submodules, a task decomposed into
subtasks, etc.
- IMPLEMENT: <semantic> -IMPLEMENT-> <code>: A task is
implemented by a function, a system module is implemented
by a set of functions, etc.

#####
output:
(esp_spp_init<|>USE<|>esp_spp_mode_t)##
(esp_spp_init<|>USE<|>esp_err_t)##
(Initializing SPP module<|>IMPLEMENT<|>esp_spp_init)##
<|COMPLETE|>

#####
-Real Data-
#####
-Input Document-
{{document}}
-Entity Set-
{{entity_set}}
#####
output:

```

Fig. A3. Prompt for relationship extraction.

```

You are an embedded system developer skilled in domain
specific customization. You are knowledgeable about general
domain knowledge about RTOS, MCU, etc.

-Goal-
Given a list of technical documents (each is an API
Reference or Datasheet for a specific module) and a user
query, first identify which documents may contain the
answer to the query. Then, write a subquery for each
candidate document to find the answer to the user query.

#####
- Example -
Document List:
["stm32_api_reference", "DHT11_datasheet", "OLED_datasheet"]

User Query:
How to read the temperature of DHT11 sensor through GPIO?

- Output -
{
  "candidate_documents": ["stm32_api_reference",
"DHT11_datasheet"],
  "sub_queries": [
    {
      "document": "stm32_api_reference",
      "sub_query": "How to read data from a GPIO
pin?"
    },
    {
      "document": "DHT11_datasheet",
      "sub_query": "How to read the temperature data
from the sensor?"
    }
  ]
}

#####
-Real Data-
#####
Document List:
{{document_list}}

User Query:
{{user_query}}

```

Fig. A4. Prompt for query decomposition.

Received 30 September 2024; revised 28 February 2025; accepted 16 April 2025